JAVA PROGRAMMING PRACTICES

Bank Account Management System

1. Aim / Objective

To design and implement a Bank Account Management System using Java classes and objects. The system models Account, SavingsAccount and CurrentAccount and performs deposit, withdrawal, interest calculation and transfer operations.

2. Algorithm

1. Start the program.
2. Create the Account base class with account number, balance and account type.
3. Create SavingsAccount and CurrentAccount classes by extending Account.
4. Define deposit, withdrawal, interest calculation and transfer methods.
5. Create objects for savings and current accounts.
6. Perform deposit, withdrawal, interest and transfer operations.
7. Display the account details and transaction results.
8. Stop the program.

3. UML Diagram

```
                    +----------------------+
                    |       Account        |
                    +----------------------+
                    | - accountNumber      |
                    | - balance            |
                    | - accountType        |
                    +----------+-----------+
                               |
                 +-------------+-------------+
                 |                           |
      +---------v----------+       +--------v---------+
      |   SavingsAccount   |       |   CurrentAccount |
      +--------------------+       +------------------+
      | - interestRate     |       | - overdraftLimit |
      +--------------------+       +------------------+
      | + calculateInterest|       | + withdraw()     |
      +--------------------+       +------------------+
```

4. Source Code

```
class Account {
    int accountNumber;
    double balance;
    String accountType;

    Account(int accountNumber, double balance, String accountType) {
        this.accountNumber = accountNumber;
        this.balance = balance;
        this.accountType = accountType;
    }

    void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: " + amount);
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient balance");
        }
```

```
    }

    void transfer(Account receiver, double amount) {
        if (amount <= balance) {
            balance -= amount;
            receiver.balance += amount;
            System.out.println("Transferred: " + amount);
        } else {
            System.out.println("Insufficient balance for transfer");
        }
    }

    void display() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Account Type: " + accountType);
        System.out.println("Balance: " + balance);
    }
}

class SavingsAccount extends Account {
    double interestRate;

    SavingsAccount(int number, double balance, double rate) {
        super(number, balance, "Savings");
        interestRate = rate;
    }

    void calculateInterest() {
        double interest = balance * interestRate / 100;
        balance += interest;
        System.out.println("Interest: " + interest);
    }
}

class CurrentAccount extends Account {
    double overdraftLimit;

    CurrentAccount(int number, double balance, double limit) {
        super(number, balance, "Current");
        overdraftLimit = limit;
    }

    @Override
    void withdraw(double amount) {
        if (amount <= balance + overdraftLimit) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Overdraft limit exceeded");
        }
    }
}

public class BankAccountManagement {
    public static void main(String[] args) {
        SavingsAccount savings =
            new SavingsAccount(101, 10000, 4);
        CurrentAccount current =
            new CurrentAccount(102, 5000, 2000);

        System.out.println("SAVINGS ACCOUNT");
        savings.deposit(2000);
        savings.withdraw(1000);
        savings.calculateInterest();
        savings.display();

        System.out.println("\\nCURRENT ACCOUNT");
        current.deposit(1000);
        current.withdraw(6500);
        current.display();

        System.out.println("\\nTRANSFER");
        savings.transfer(current, 2000);

        System.out.println("\\nFINAL DETAILS");
        savings.display();
        current.display();
    }
}
```

5. Output Screenshot

Actual terminal-style screenshot of the compilation and execution output:

```
C:\Java> javac BankAccountManagement.java
C:\Java> java BankAccountManagement

SAVINGS ACCOUNT
Deposited: 2000.0
```

```
Withdrawn: 1000.0
Interest: 440.0
Account Number: 101
Account Type: Savings
Balance: 11440.0

CURRENT ACCOUNT
Deposited: 1000.0
Withdrawn: 6500.0
Account Number: 102
Account Type: Current
Balance: -500.0

TRANSFER
Transferred: 2000.0

FINAL DETAILS
Account Number: 101
Account Type: Savings
Balance: 9440.0
Account Number: 102
Account Type: Current
Balance: 1500.0

C:\Java>
```

6. Compilation and Execution Commands

```
javac BankAccountManagement.java
java BankAccountManagement
```

7. Result

Thus, the Bank Account Management System was successfully implemented using Java classes and objects. Deposit, withdrawal, interest calculation and transfer operations were executed successfully.